AI600 Deep Learning

Assignment #1

Due: Tuesday February 17, 2026

Professor:	Prof. Dr. Momin Uppal
TA(s):	Muhammad Aleem Shakeel, Muhammad Usman, Muhammad Qasim Ayub, Usman Ahad
Student:	Faaiz Khan
Roll No.:	25280051

Honor Pledge:

I affirm that I have abided by the academic integrity policy. I have neither given nor received unauthorized aid on this assignment.

Question 1: Feedforward Neural Networks Using Tabular Data

Part A: Data Preprocessing and Exploratory Analysis

The dataset can be described — including the number of samples as well as the number of features and their data types — using the `info` function provided by the `pandas` library.

```
<class 'pandas.DataFrame'>
RangeIndex: 41348 entries, 0 to 41347
Data columns (total 7 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   neighbourhood_group  40509 non-null  str
 1   room_type            40737 non-null  str
 2   minimum_nights       40026 non-null  float64
 3   amenity_score        40432 non-null  float64
 4   number_of_reviews    40225 non-null  float64
 5   availability_365     40753 non-null  float64
 6   price_class          41348 non-null  int64
dtypes: float64(4), int64(1), str(2)
memory usage: 2.2 MB
```

Figure 1: The output of the `info` function describing the dataset.

Once the data is understood, the next priority is to handle the missing values in the dataset appropriately.

Table 1: Data Imputation Strategy for Each Feature in the Dataset

Feature	**Strategy**	**Rationale**
`neighbourhood_group`	Deletion	Dropping these rows avoids introducing noise into categorical spatial data. With over 40k samples remaining, the impact on dataset size is negligible.
`room_type`	Deletion	Prevents synthetic bias in a primary feature. Deletion is preferred over mode imputation to maintain the integrity of the categorical distribution.
`minimum_nights`	Median	Accounts for the "semi-categorical" nature of integer-based night stays and provides robustness against extreme outliers (skewed data).
`amenity_score`	Mean	Captures the central tendency of the scores. Since scores are typically normally distributed, the mean reflects the most representative value.
`number_of_reviews`	Constant (0)	Missing values likely indicate a new listing with no historical activity; a constant of zero is the most logical factual replacement.
`availability_365`	Median	Protects against high variance in listing availability, ensuring the "typical" availability is captured rather than an average skewed by unbooked properties.

Now that the data is clean, analysing the class distribution is an important part of EDA that may lead to valuable insights.

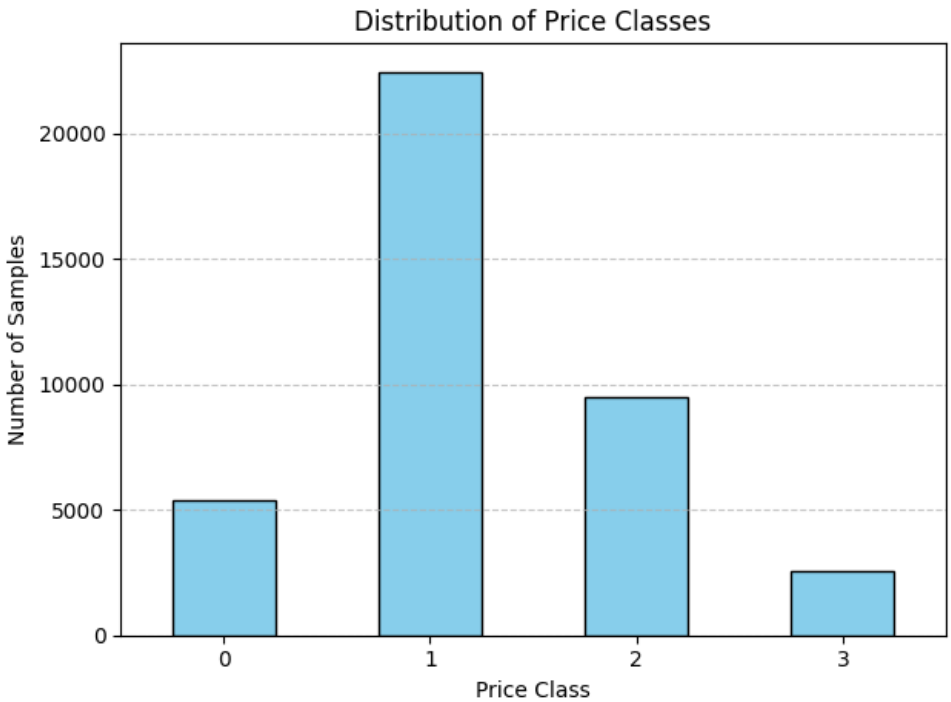

Figure 2: A visualisation of the class distribution in the dataset.

Table 2: Distribution of the Target Variable (`price_class`)

Price Class	**Sample Count**	**Percentage (%)**
Class 0 (Budget)	5,372	13.46
Class 1 (Moderate)	22,477	56.33
Class 2 (Premium)	9,502	23.81
Class 3 (Luxury)	2,554	6.40

From the visualisations above, the imbalance is obvious and unsurprising: the 'Moderate' class takes up 56.33% of all the listings in the dataset.
The next port of call is to encode the categorical variables using an appropriate encoding scheme. The one chosen for this dataset — specifically the `neighbourhood_group` and `room_type` variables — is 'One-Hot Encoding'. The reason to use this scheme is twofold:

- These variables have no mathematical 'ranking'. For example, Manhattan cannot be higher than Brooklyn in a way that a computer can naturally calculate without bias.
- Since the number of unique categories is small: 5 and 3 respectively, we can use this scheme with a much less risk of falling into the 'curse of dimensionality'.

The next step is is to normalize the numerical features. The method used for this dataset is Standardisation (Z-score Normalisation). Simply put, using the formula $z = \frac{x-\mu}{\sigma}$, this method shifts the data so the mean (μ) is 0 and the standard deviation (σ) is 1. There are a few reasons to use this method:

- It ensures that algorithms do not get biased by the sheer magnitude of the number.
- It allows for faster convergence without wild oscillations in models that use Gradient Descent.
- It handles outliers much better than other methods like Min-Max Scaling.

Once the data is cleaned, split, and preprocessed; it is important to get a basic understanding of the nature of the data and whether there are any obvious identifiable patterns/consistencies in it. For the categorical features, this is done by creating a proportionate stacked bar chart.

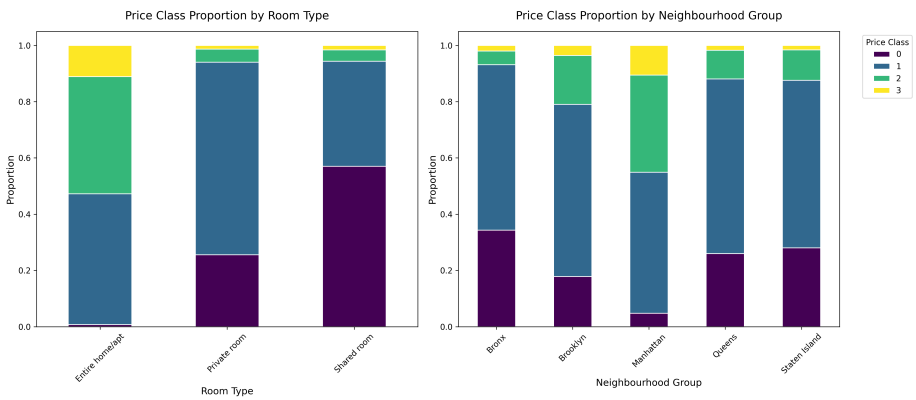

Figure 3: A visualisation of the relationship between the categorical features and the label.

For the numerical features, this is done using box plots.

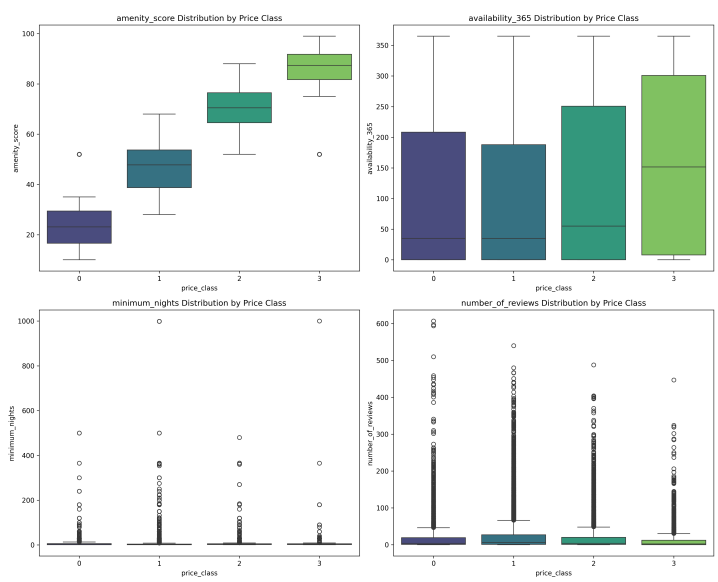

Figure 4: A visualisation of the relationship between the numerical features and the label.

Understanding the correlation between features is also an important part of EDA. The correlation matrix can be visualised using a heatmap.

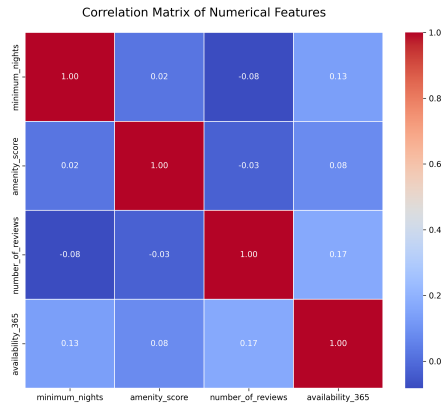

Figure 5: Correlation matrix of numarical features visualised in a heatmap.

From all the available information above, there are two features that stick out: `amenity_score` and `room_type`. Looking at the bar chart in Figure 3, it is apparent that more than 90% of listings in class 2 and 3 fall under 'Entire home/apartment' room type. Additionally, looking at the box plots in Figure 4, the amenity score almost perfectly seperates each price class. Therfore, it stands to reason that the amenity score feature will be both unusually predictive and suspiciously dominant.

Part B(a): Two-Layer Perceptron Implemented from Scratch

A two-layer perceptron was impemented from scratch using only `NumPy` operations. After programming the `FFNN` class and defining a `train` function that performed all the necessary steps required to train a simple neural network, there were still a lot of issues/obstacles that needed to be ironed out. There were multiple tweaks and trial runs before the parameters were set on something that worked for the required task at hand. These are listed below:

1. **The size of the hidden layers:** After multiple trial runs, it was apparent that the required number of neurons in the hidden layer did not need to be an extravagant amount. At the end, each hidden layer ended up with 16 neurons.

2. **The number of iterations:** Since the learning rate was kept constant (0.1) throughout the training experiments, the number of iterations needed to be sure that both models would ultimately converge was 1000.

3. **The initialisation of the weights:** After running the training with random initialisation, the models kept falling into the vanishing gradient problem. To avoid this, after some research online, 'He' initialisation was used for the ReLU model whereas 'Xavier' initialisation was used for the sigmoid model.

Once all the details were satisfactory, a final training was done which returned the following accuracies:

Table 3: Final Training and Validation Accuracies for the Sigmoid and ReLU Models

Model	Training Accuracy (%)	Validation Accuracy (%)
Sigmoid	77.29	77.07
ReLU	83.36	83.10

The plot for the gradients of both the models is also shown below.

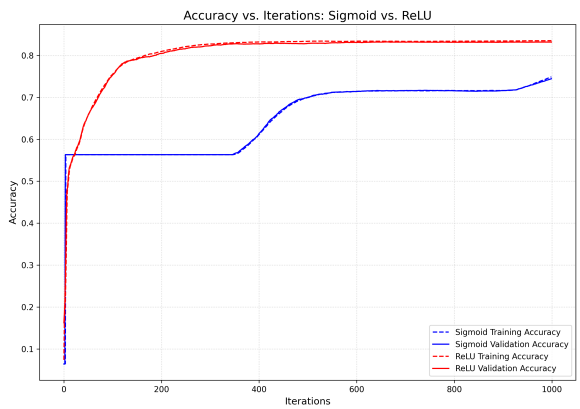

Figure 6: The accuracies of the models as a function of the number of iterations.

Part B(b): Gradient Magnitude Comparison Across Layers

To analyze the internal learning dynamics and the "health" of the gradient flow, the average absolute magnitude of the gradients for the first hidden layer weights ($\delta W^{(1)}$) and the second hidden layer weights ($\delta W^{(2)}$) were tracked over 1000 iterations.

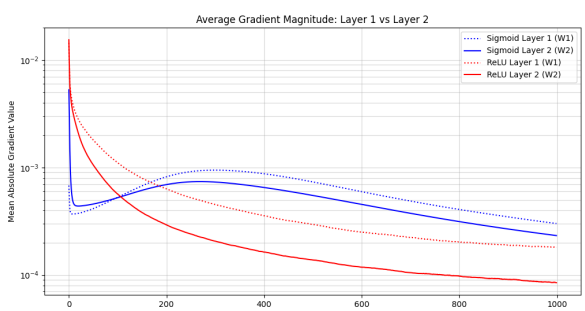

Figure 7: The average gradient magnitude of the hidden layers as a function of the number of iterations.

The following trends were observed during the optimisation process for both activation functions:

- The sigmoid gradient magnitudes exhibited a distinct curve. Initially, magnitudes dropped sharply, followed by a temporary increase as weights moved into the active regions of the

Sigmoid curve, and finally, a steeper decrease toward the end of the iterations. Notably, the gradients for Layer 1 were consistently slightly larger but decreased and increased more aggressively than Layer 2 at the start of the training.

- The gradients for ReLU started with significantly higher magnitudes than Sigmoid. While they decreased over time, the decrease was gradual and eventually plateaued. In this network, Layer 1 gradient magnitudes remained slightly higher than Layer 2, but both layers moved with stable, non-vanishing signals.

These results reveal several insights into the optimization of deep neural networks:

- The slightly steeper final decline in Sigmoid gradients compared to ReLU might be demonstrative of the vanishing gradient problem. Although the decline is not extreme in this specific architecture, the multiplicative effect of the Sigmoid derivative (capped at 0.25) inherently attenuates the error signal more than ReLU's constant 1.0 derivative.
- Sigmoid's initial drop followed by the surge is indicative weights transitioning through different regions of the S-curve; the final steady decrease suggests the model is approaching a region of lower sensitivity (saturation). ReLU's more gradual decrease indicates a more stable and preserved gradient flow throughout the training process.
- In both networks, Layer 1 gradients are larger than Layer 2 which is usually not the case when it comes to neural networks.
- ReLU's and Sigmoid's plateau signifies 'successful' convergence where the error $(\hat{y} - y)$ is minimized and updates become stable.

Part C(a): Gradient-Based Feature Attribution (Analytical)

In the architecture studied, the forward pass begins with the input $x = x^{(0)}$. The relationship between the input and the first hidden layer's pre-activation $a^{(1)}$ is defined as:

$$a^{(1)} = W^{(1)}x^{(0)} + b^{(1)}$$

This is important to point out since the the loss $\mathcal{L}$ only depends on $\mathbf{x}$ through the first layer pre-activations. To find $\frac{\partial \mathcal{L}}{\partial \mathbf{x}}$, the chain rule is applied:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial a^{(1)}} \cdot \frac{\partial a^{(1)}}{\partial \mathbf{x}}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial a^{(1)}} \cdot (W^{(1)})^\top$$

The notes also say that the first term on the right hand side of the above equation equals $\vec{\delta}^{(1)}$ which is obtained through the standard backward pass recursion.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = (W^{(1)})^\top \cdot \vec{\delta}^{(1)}$$

The pseudocode (algorithmic steps) that outlines the analytical computation of gradients with respect to each input feature is provided below as well as how the magnitude of these gradients can be used to rank feature importance.

Algorithm 1 Gradient-Based Feature Attribution

```
procedure FEATURE_ATTRIBUTION({x⃗_k, y_k}_{k=1}^N, W, b⃗)
    Initialize importance vector I⃗ of size M
    for k = 1 to N do
        FORWARDPASS(x)
        δ⃗^(L) ← ∂L/∂x⃗^(L) ⊙ g'_L(a⃗^(L))
        for l = L down to 2 do
            δ⃗^(l−1) ← (W^(l)⊤ δ⃗^(l)) ⊙ g'(a⃗^(l−1))
        end for
        G⃗^(k) ← W^(1)⊤ δ⃗^(1)
        for i = 1 to M do
            I⃗_i ← I⃗_i + (1/N)|G_i^(k)|
        end for
    end for
    return the importance vector sorted descendingly
end procedure
```

To justify this from an optimisation perspective, the magnitude $\left|\frac{\partial \mathcal{L}}{\partial \mathbf{x}}\right|$ is a meaningful measure because it represents the sensitivity of the network's loss to each feature. This implies that features with larger gradient magnitudes are those that have the most significant influence on the loss function, identifying them as the most 'informative' inputs for the model's current state.

Part C(b): Implementation and Results

To implement this feature attribution in the current architecture of the FFNN that was created, the simple solution is to just add a line of code to the backpropagation algorithm and then create a function that determines the feature relevance by aggregating the absolute magnitude of input gradients exclusively over correctly classified training instances. This approach ensures that the identified influential features are those that contribute to the model's successful decision-making process. Since the backpropagation algorithm already calculates all the required values for the calculation, the importance vector containing the feature gradients can simply be calculated and stored in it. Once the training was done, the results are quite similar to what was expected in Part A.

Table 4: Ranked Influential Features based on Gradient Sensitivity (Sigmoid Network)

Rank	Feature Name	Sensitivity Magnitude
1	`num__amenity_score`	0.417094
2	`cat__room_type_Private room`	0.109173
3	`cat__neighbourhood_group_Manhattan`	0.068333
4	`cat__neighbourhood_group_Queens`	0.042537
5	`cat__neighbourhood_group_Staten Island`	0.033062
6	`cat__room_type_Shared room`	0.026062
7	`cat__neighbourhood_group_Brooklyn`	0.023719
8	`num__availability_365`	0.018158
9	`num__minimum_nights`	0.015609
10	`num__number_of_reviews`	0.006254

Table 5: Ranked Influential Features based on Gradient Sensitivity (ReLU Network)

Rank	Feature Name	Sensitivity Magnitude
1	`num__amenity_score`	0.634728
2	`cat__room_type_Shared room`	0.260162
3	`cat__room_type_Private room`	0.166986
4	`cat__neighbourhood_group_Staten Island`	0.147451
5	`cat__neighbourhood_group_Manhattan`	0.137340
6	`cat__neighbourhood_group_Queens`	0.125447
7	`cat__neighbourhood_group_Brooklyn`	0.110445
8	`num__minimum_nights`	0.074399
9	`num__availability_365`	0.074173
10	`num__number_of_reviews`	0.046926

In both architectures, `num__amenity_score` emerges as the primary driver of the model's output, exhibiting the highest aggregate sensitivity. The significantly higher sensitivity magnitudes observed in the ReLU network are attributable to ReLU's constant gradient of 1.0, which avoids the vanishing gradient decay inherent in Sigmoid's 0.25-capped derivative. While both networks align with the general findings of Part A, the ReLU network identifies `cat__room_type_Shared room` as more influential than the Sigmoid network suggests. This shift indicates that the ReLU model has established a more sensitive decision boundary around room-sharing features, capturing non-linear relevance that was secondary in the Sigmoid analysis.

The implemented algorithm is very similar to the one derived in Part C(a), as it uses the training cycle to store the feature attributes. However, the main difference is that the aggregation of the absolute gradient magnitudes is restricted specifically to correctly classified training instances.

Algorithm 2 Gradient-Based Feature Attribution (According to Section 2.8 of Charu C. Aggarwal's Textbook)

```
procedure FEATURE_ATTRIBUTION({x⃗_k, y_k}_{k=1}^N, W, b⃗)
    Initialize importance vector I⃗ of size M
    N_correct ← 0
    for k = 1 to N do
        FORWARDPASS(x⃗_k)
        if argmax(ŷ⃗_k) == argmax(y⃗_k) then
            N_correct ← N_correct + 1
            δ⃗^(L) ← ŷ⃗_k − y⃗_k
            for l = L down to 2 do
                δ⃗^(l−1) ← (W^(l)⊤ δ⃗^(l)) ⊙ g′(a⃗^(l−1))
            end for
            G⃗^(k) ← W^(1)⊤ δ⃗^(1)
            for i = 1 to M do
                I⃗_i ← I⃗_i + |G_i^(k)|
            end for
        end if
    end for
    I⃗ ← I⃗/N_correct
    return the importance vector sorted descendingly
end procedure
```

Part D: Test Evaluation and Generalisation Analysis

The trained models were evaluated on the provided test dataset. The reported accuracies are as follows:

- **Sigmoid Network Test Accuracy:** 40.73%
- **ReLU Network Test Accuracy:** 34.40%

The table below illustrates the performance gap between the datasets:

Table 6: Model Accuracy Comparison across Datasets

Model	Training Acc. (%)	Validation Acc. (%)	Test Acc. (%)
Sigmoid	77.29	77.07	40.73
ReLU	83.36	83.10	34.40

There is a significant generalization gap. While the models performed consistently across training and validation sets (with less than 1% difference), they failed to generalize to the test set, with accuracy dropping by approximately 36% for Sigmoid and 49% for ReLU. This indicates that while the models did not "overfit" to the training noise, they failed to learn features that remain invariant across different data distributions.
The failure in generalization can be traced back to the following evidence:

- **Gradient-Based Attribution (Part C):** The analysis based on Section 2.8 of the textbook revealed that the models are hyper-sensitive to the `num__amenity_score` feature. In the ReLU network, the sensitivity magnitude for this feature (0.634) was substantially higher than other features.
- **Exploratory Data Analysis (Part A):** The model has effectively become an "amenity score classifier" rather than a robust price predictor. If the distribution of amenity scores or their relation to price classes differs in the test set compared to the training set, the model's over-reliance on this single feature leads to categorical failure.

The model performs well during training because it identifies a strong mathematical correlation between the amenity score and the target within that specific data slice. However, by assigning such high sensitivity to one attribute, the model ignores the secondary predictive power of other features. As specified in the textbook, backpropagation identifies the most relevant features for classification; in this case, it highlights an unhealthy dependency on a single variable that does not hold its predictive power in the test environment.
A strategy that could mitigate this is the Implementation of L2 regularization within the back-propagation update. L2 regularization penalizes large weight magnitudes. By adding this penalty, the model would be prevented from assigning the disproportionately high weights to the `num__amenity_score` input that were observed in the gradient analysis. This forces the network to distribute its learned importance across multiple features, resulting in a more balanced decision boundary that is less susceptible to shifts in any single feature's distribution.

Question 2: Bias Gradients, Parameter Sharing, and Activation Effects

The computation graph for the FFNN in this question is shown in the image below:

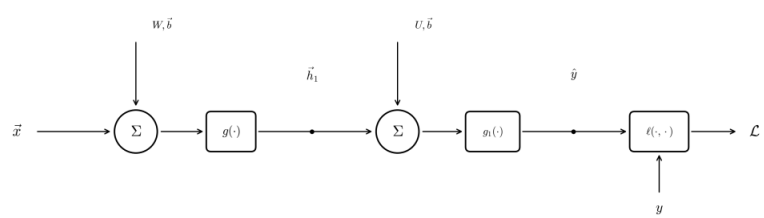

Figure 8: Computation graph for a two-layer MLP with a shared bias parameter

The paths where the shared bias vector b affects the loss function can be outlined by looking at the computation graph and the forward pass equations provided.
The first path, where the bias directly influences the pre-activation of the output layer:

$$L = \ell(g_1(U\mathbf{h}_1 + b), y)$$

The second path, where the bias influences the loss by first changing the representation in the hidden layer, which then flows through the rest of the network.

$$L = \ell(g_1(U(g(W\mathbf{x} + b)) + b), y)$$

In this FFNN, when a parameter is used multiple times in the forward pass, its total gradient is the sum of the gradients calculated for each occurrence. From this, the backpropagation required utilises the chain rule as follows:

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial b} + \frac{\partial L}{\partial a^{(1)}} \cdot \frac{\partial a^{(1)}}{\partial b}$$

Starting with the direct path:

$$\frac{\partial L}{\partial a^{(2)}} = \frac{\partial L}{\partial y} \odot g_1'(a^{(2)}) = \vec{\delta}^{(2)}$$

$$\frac{\partial a^{(2)}}{\partial b} = I$$

Continuing with the 'indirect' path:

$$\frac{\partial L}{\partial a^{(1)}} = (U^\top \vec{\delta}^{(2)}) \odot g_1'(a^{(1)}) = \vec{\delta}^{(1)}$$

$$\frac{\partial a^{(1)}}{\partial b} = I$$

Putting the two together:

$$\frac{\partial L}{\partial b} = \vec{\delta}^{(2)} + \vec{\delta}^{(1)}$$

Or, in terms of $\vec{\delta}^{(2)}$:

$$\frac{\partial L}{\partial b} = \vec{\delta}^{(2)} + (U^\top \vec{\delta}^{(2)}) \odot g_1'(a^{(1)})$$